

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DO CEARÁ – IFCE**

Cursos de Engenharia de Computação e de Telecomunicações

Disciplina: Cálculo Numérico

VINÍCIUS MENEZES MONTE

MARCELO FLÁVIO DE CARVALHO FROTA PORTO

**CÁLCULO NUMÉRICO E CIÊNCIA DA COMPUTAÇÃO:
história e fundamentos da Pseudoinversa de Moore-Penrose**

Fortaleza – Ceará

2026

VINÍCIUS MENEZES MONTE
MARCELO FLÁVIO DE CARVALHO FROTA PORTO

**CÁLCULO NUMÉRICO E CIÊNCIA DA COMPUTAÇÃO:
história e fundamentos da Pseudoinversa de Moore-Penrose**

Trabalho apresentado à disciplina de Cálculo Numérico dos cursos de Engenharia de Computação e de Telecomunicações do Instituto Federal do Ceará (IFCE), como requisito parcial de avaliação.

Orientador: Prof. Dr. Francisco Aquino.

SUMÁRIO

Introdução	3
1 História do cálculo numérico e sua relação com a computação	4
1.1 Marcos históricos: da Antiguidade ao século XIX	4
1.2 O século XX e o surgimento dos computadores digitais	5
1.3 A era dos computadores pessoais e o cálculo numérico atual	5
2 Pseudoinversa de Moore-Penrose	7
2.1 Contexto histórico	7
2.2 Fundamentação teórica	7
2.2.1 Definição e as quatro condições de Moore-Penrose	7
2.2.2 Cálculo pela decomposição em valores singulares	8
2.2.3 Sistemas sobredeterminados: mínimos quadrados	8
2.2.4 Sistemas subdeterminados: mínima norma	8
2.2.5 Número de condição e exemplo manual	9
2.3 Algoritmo e implementação computacional	9
2.3.1 Pseudocódigo	9
2.3.2 Implementação em Python	10
2.4 Resultados e análise	10
2.4.1 Validação	10
2.4.2 Exemplo 1 – sistema sobredeterminado (4×2)	11
2.4.3 Exemplo 2 – sistema subdeterminado (2×4)	11
2.4.4 Análise de posto e erro: SVD truncada	11
2.4.5 Análise do número de condição	12
2.5 Aplicações em Engenharia de Computação e Telecomunicações	12
2.5.1 Computação Gráfica: ajuste de curvas e compressão por SVD	12
2.5.2 Telecomunicações: estimação de canal MIMO	13
2.5.3 Aprendizado de máquina: regressão linear	13
3 Reflexão: cálculo numérico, computação e engenharia	15
Conclusão	16
Referências	17

Introdução

O cálculo numérico pode ser definido como o conjunto de métodos e algoritmos que permitem resolver, de forma aproximada e por meio de máquinas digitais, problemas matemáticos que raramente possuem solução analítica fechada. Como observa Aquino (2020), trata-se de uma disciplina de cunho eminentemente prático, voltada a engenheiros e cientistas que usam os métodos numéricos como ferramenta para a resolução de problemas reais. Equações não lineares, sistemas lineares de grande porte, integração de funções sem primitiva conhecida e ajuste de modelos a dados experimentais são situações cotidianas em que a resposta exata é inacessível e a aproximação controlada torna-se indispensável.

Para a Engenharia de Computação e de Telecomunicações, esse ferramental faz parte do dia a dia. Simulação de circuitos, processamento de sinais, codificação de canal, otimização de redes e, mais recentemente, aprendizado de máquina apoiam-se todos em operações numéricas sobre matrizes e vetores. Há aí também uma relação histórica: o cálculo numérico amadureceu junto com a ciência da computação. Foi a necessidade de resolver problemas grandes que motivou a construção das primeiras máquinas digitais, e foram essas máquinas que tornaram viáveis métodos antes impossíveis de fazer à mão.

Objetivo geral. Analisar a história do cálculo numérico e sua relação com a ciência da computação, aprofundando um tópico específico da disciplina: a *pseudoinversa de Moore-Penrose*.

Objetivos específicos. (i) descrever marcos históricos relevantes do cálculo numérico; (ii) relacionar o desenvolvimento de métodos numéricos com a evolução de algoritmos e hardware; (iii) estudar em profundidade a pseudoinversa, sua fundamentação, algoritmo e implementação; e (iv) discutir suas aplicações em Engenharia de Computação e Telecomunicações.

A pseudoinversa foi escolhida por um motivo concreto. Ela estende a ideia de inversa para os casos em que a inversa clássica não existe (matrizes retangulares ou singulares) e, de uma só vez, dá a solução de mínimos quadrados dos sistemas sobredeterminados e a solução de mínima norma dos subdeterminados. É um tema que liga a Álgebra Linear ao cálculo numérico e que aparece com frequência na Computação Gráfica que estudo, no ajuste de curvas e na compressão de imagens por decomposição em valores singulares (SVD).

O Capítulo 1 traça um panorama histórico do cálculo numérico e de sua ligação com a computação. O Capítulo 2 trata da pseudoinversa em detalhe: contexto histórico, fundamentação teórica, o algoritmo baseado na SVD, a implementação, os resultados e as aplicações em engenharia. O Capítulo 3 reflete sobre a relação entre cálculo numérico, computação e engenharia, e a Conclusão retoma os objetivos do trabalho.

1 História do cálculo numérico e sua relação com a computação

Este capítulo traça um panorama histórico do cálculo numérico, com atenção à evolução dos métodos e à sua ligação com a ciência da computação. Há um padrão que se repete: quase toda técnica numérica nasceu de uma necessidade prática de cálculo e logo esbarrou no limite do que uma pessoa consegue computar à mão. Esse limite só foi vencido com as máquinas.

1.1 Marcos históricos: da Antiguidade ao século XIX

A ideia de aproximar grandezas que não se conseguia calcular exatamente é antiga. No século III a.C., Arquimedes estimou o valor de π inscrevendo e circunscrevendo polígonos regulares a um círculo, estreitando o intervalo entre os perímetros à medida que aumentava o número de lados – um procedimento de refinamento sucessivo que é, em essência, um método numérico. Civilizações babilônica, chinesa, indiana e árabe desenvolveram algoritmos para extrair raízes e resolver equações; o tratado de al-Khwarizmi (século IX) dá nome ao próprio termo *algoritmo*. Métodos que ainda hoje estudamos têm raízes igualmente antigas: a eliminação gaussiana para sistemas lineares já aparece, em forma embrionária, no tratado chinês *Nove Capítulos sobre a Arte Matemática* (por volta do século II a.C.), e algoritmos iterativos para extrair raízes quadradas eram conhecidos pelos babilônios, precursores diretos do método de Newton.

Entre os séculos XVII e XIX, o desenvolvimento do cálculo diferencial e integral por Newton e Leibniz forneceu a base teórica sobre a qual floresceram os métodos numéricos modernos. Surgiram técnicas que ainda hoje integram qualquer curso da disciplina: o método de Newton-Raphson para raízes de equações, as fórmulas de interpolação de Newton e de Lagrange, a quadratura para integração e o método dos mínimos quadrados, formalizado por Gauss e Legendre no início do século XIX para o ajuste de observações astronômicas. É justamente dessa tradição dos mínimos quadrados que, mais tarde, emergiria a pseudoinversa. Nesse período, a “computação” era manual e apoiava-se em extensas tabelas de logaritmos e funções trigonométricas, laboriosamente calculadas e publicadas. Foi também nesse espírito que Charles Babbage concebeu, na década de 1820, a *Máquina Diferencial*, projetada para automatizar exatamente esse tipo de tabulação por diferenças finitas, uma antecipação mecânica da computação que só se concretizaria um século depois.

Muitos dos métodos que estruturam a disciplina de Cálculo Numérico amadureceram ao longo desse percurso. A eliminação gaussiana e suas variantes (as decomposições LU e de Cholesky) sistematizaram a resolução de sistemas lineares; métodos iterativos como os de Jacobi e de Gauss-Seidel deram alternativas para sistemas grandes; e técnicas para autovalores, como o método da potência e, mais tarde, o algoritmo QR, prepararam o terreno para a decomposição em valores singulares, a ferramenta que se mostrará central no Capítulo 2.

1.2 O século XX e o surgimento dos computadores digitais

Na primeira metade do século XX, antes dos computadores eletrônicos, cálculos de grande porte para balística, meteorologia e física nuclear eram realizados por equipes de *computadores humanos* – pessoas que executavam contas com lápis, tabelas e calculadoras mecânicas e eletromecânicas. A Segunda Guerra Mundial acelerou enormemente essa demanda, tornando evidente a necessidade de automatizar o cálculo.

Um exemplo do novo patamar de cálculo que as máquinas tornaram possível é o método de Monte Carlo. Desenvolvido por Stanislaw Ulam, John von Neumann e Nicholas Metropolis no Projeto Manhattan, ao final da década de 1940, ele simulava a difusão de nêutrons por amostragem aleatória. É uma técnica que só faz sentido com bastante poder de cálculo, e Aquino (2020) a apresenta no capítulo final de seu livro, como exemplo de como a simulação numérica ampliou o alcance da engenharia.

A resposta veio com as primeiras máquinas digitais: o ABC, o Colossus e, sobretudo, o ENIAC (1945), além do modelo teórico da máquina universal de Alan Turing. Desde o início, três coisas andaram juntas: a necessidade de resolver numericamente equações diferenciais e sistemas lineares grandes, o desenvolvimento de algoritmos eficientes e o crescimento da computação como área própria. Nomes como von Neumann, Turing e Grace Hopper estão no centro dessa história. Em 1947, von Neumann e Goldstine publicaram um dos primeiros estudos sistemáticos sobre erros de arredondamento na resolução de sistemas lineares, o que ajudou a firmar a *análise numérica* como campo de pesquisa; mais tarde, James Wilkinson aprofundaria a análise de estabilidade.

1.3 A era dos computadores pessoais e o cálculo numérico atual

A popularização dos computadores pessoais e das estações de trabalho, a partir das décadas de 1970 e 1980, levou o cálculo numérico para fora dos grandes centros de pesquisa. Linguagens e ambientes voltados à computação científica se multiplicaram: Fortran, C, C++, Pascal e, depois, ambientes interativos como o MATLAB (criado por Cleve Moler a partir das bibliotecas LINPACK e EISPACK), além de Scilab e Python com as bibliotecas NumPy e SciPy. Por trás desses ambientes, bibliotecas numéricas altamente otimizadas como BLAS e LAPACK passaram a constituir a infraestrutura sobre a qual praticamente toda a engenharia computacional moderna se apoia.

Nem todo avanço veio do hardware; alguns dos mais importantes foram algorítmicos. O caso mais conhecido é a Transformada Rápida de Fourier (FFT), popularizada por Cooley e Tukey em 1965, que baixou o custo da transformada discreta de Fourier de $O(n^2)$ para $O(n \log n)$. Sozinho, esse algoritmo tornou prático o processamento digital de sinais em tempo real e virou peça central das telecomunicações, da compressão de áudio e imagem a esquemas de modulação como o OFDM. Curiosamente, foi também em 1965 que Golub e Kahan publicaram o algoritmo

estável para a SVD.

O livro-texto desta disciplina (AQUINO, 2020) é um bom exemplo dessa cultura: usa o Scilab como ferramenta e prefere uma abordagem prática, apresentando cada método junto aos recursos que o tornam aplicável. O que importa reter é que o avanço da computação permitiu usar em larga escala métodos antes impraticáveis à mão. A pseudoinversa ilustra bem isso: sua definição é de 1920, mas ela só passou a ser usada de forma corrente depois do algoritmo estável de SVD, de Golub e Kahan (1965). É esse o tema do próximo capítulo.

2 Pseudoinversa de Moore-Penrose

2.1 Contexto histórico

A pseudoinversa, ou inversa generalizada, responde a uma pergunta natural: se a inversa clássica A^{-1} só existe para matrizes quadradas e não singulares, o que fazer quando A é retangular ou singular? A semente do conceito aparece já em 1903, com Fredholm, no estudo de operadores integrais. A primeira formulação algébrica explícita é de Eliakim Hastings Moore, que em 1920 apresentou à *American Mathematical Society* a noção de “recíproca geral” de uma matriz qualquer (MOORE, 1920). O trabalho de Moore, contudo, empregava notação pouco difundida e permaneceu relativamente obscuro por décadas.

O conceito foi redescoberto de forma independente em problemas aplicados, primeiro por Bjerhammar, na geodésia, por volta de 1951, e depois por Roger Penrose, em 1955. Penrose mostrou que, para toda matriz A , existe uma *única* matriz A^+ que satisfaz quatro equações simples, hoje chamadas de condições de Moore-Penrose (PENROSE, 1955). Essa caracterização reuniu as abordagens anteriores e fixou o nome “pseudoinversa de Moore-Penrose”. Faltava ligar a teoria à prática, e foi a decomposição em valores singulares que fez isso: o algoritmo estável de Golub e Kahan (1965), depois refinado por Golub e Reinsch, tornou o cálculo de A^+ confiável até para matrizes mal condicionadas, o que a firmou como ferramenta padrão da computação científica (GOLUB; VAN LOAN, 2013).

2.2 Fundamentação teórica

2.2.1 Definição e as quatro condições de Moore-Penrose

Dada uma matriz $A \in \mathbb{R}^{m \times n}$, a pseudoinversa $A^+ \in \mathbb{R}^{n \times m}$ é a única matriz que satisfaz as quatro condições:

$$(1) AA^+A = A; \quad (2) A^+AA^+ = A^+; \quad (3) (AA^+)^T = AA^+; \quad (4) (A^+A)^T = A^+A. \quad (2.1)$$

As duas primeiras generalizam a relação $AA^{-1}A = A$ da inversa clássica; as duas últimas exigem que os produtos AA^+ e A^+A sejam matrizes simétricas, isto é, projeções ortogonais. Quando A é quadrada e invertível, verifica-se que $A^+ = A^{-1}$, de modo que a pseudoinversa é, de fato, uma extensão do conceito de inversa.

2.2.2 Cálculo pela decomposição em valores singulares

A forma numericamente robusta de obter A^+ parte da decomposição em valores singulares (SVD). Toda matriz $A \in \mathbb{R}^{m \times n}$ admite a fatoração

$$A = U \Sigma V^T, \quad (2.2)$$

em que $U \in \mathbb{R}^{m \times m}$ e $V \in \mathbb{R}^{n \times n}$ são ortogonais e $\Sigma \in \mathbb{R}^{m \times n}$ é “diagonal”, com entradas $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ (os valores singulares) e zeros nas demais posições, onde $r = \text{posto}(A)$. A pseudoinversa é então

$$A^+ = V \Sigma^+ U^T, \quad (2.3)$$

sendo Σ^+ obtida transpondo Σ e substituindo cada valor singular não nulo σ_i por seu recíproco $1/\sigma_i$. As colunas de U e V formam bases ortonormais para os quatro subespaços fundamentais de A , e a ortogonalidade dessas matrizes é o que garante a estabilidade numérica do cálculo.

2.2.3 Sistemas sobredeterminados: mínimos quadrados

Quando $m > n$ (mais equações que incógnitas), o sistema $Ax = b$ é, em geral, incompatível. A pseudoinversa fornece a solução de *mínimos quadrados*

$$x^* = A^+ b, \quad x^* = \arg \min_x \|Ax - b\|_2. \quad (2.4)$$

Para o caso de posto coluna completo, a equação normal $A^T A x = A^T b$ conduz à forma

$$A^+ = (A^T A)^{-1} A^T, \quad (2.5)$$

exatamente a expressão apresentada por Aquino (2020, p. 105), que a denomina “pseudo-inversa ou inversa generalizada de Moore-Penrose”. Convém lembrar, porém, que a expressão (2.5) é um caso particular: ela só vale quando $A^T A$ é invertível e, mesmo assim, pode ser mal condicionada. A forma via SVD (2.3) vale para qualquer matriz, inclusive as deficientes em posto, e é mais estável; por isso é a usada nas implementações de referência.

2.2.4 Sistemas subdeterminados: mínima norma

Quando $m < n$ (menos equações que incógnitas), o sistema possui infinitas soluções. Entre todas elas, a pseudoinversa seleciona a de *mínima norma*:

$$x^* = A^+ b, \quad x^* = \arg \min \{ \|x\|_2 : Ax = b \}, \quad (2.6)$$

o que equivale a escolher a solução sem componentes no espaço nulo de A .

2.2.5 Número de condição e exemplo manual

A sensibilidade do problema a perturbações nos dados é medida pelo número de condição

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}. \quad (2.7)$$

Valores elevados de $\kappa(A)$ indicam matriz mal condicionada: pequenas perturbações em b podem produzir grandes variações em x . O próprio Aquino (2020, p. 62) ilustra o fenômeno com a equação $x^2 + 2,15x + 2,14 = 0$, cujas raízes mudam de forma amplificada quando o termo independente sofre uma pequena perturbação – exemplo clássico de mau condicionamento.

A título de exemplo manual com números pequenos, considere $A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}$, de posto coluna completo. Tem-se $A^T A = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$, com determinante 6, e $(A^T A)^{-1} = \frac{1}{6} \begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix}$. Logo,

$$A^+ = (A^T A)^{-1} A^T = \frac{1}{6} \begin{pmatrix} 5 & 2 & -1 \\ -3 & 0 & 3 \end{pmatrix} = \begin{pmatrix} 5/6 & 1/3 & -1/6 \\ -1/2 & 0 & 1/2 \end{pmatrix},$$

resultado que coincide, dígito a dígito, com o produzido pela SVD na implementação computacional da próxima seção.

2.3 Algoritmo e implementação computacional

2.3.1 Pseudocódigo

O algoritmo para o cálculo de A^+ via SVD é direto:

```
Entrada: matriz A (m x n), tolerancia relativa rcond
Saída: pseudoinversa A+ (n x m)

1. [U, s, V] <- SVD(A) // s = valores singulares
2. threshold <- rcond * s[1] // maior singular
3. para i = 1 ate r:
4.     se s[i] > threshold entao
5.         s_inv[i] <- 1 / s[i]
6.     senao
7.         s_inv[i] <- 0 // descarta singular ~ 0
8. Sigma+ <- diag(s_inv) transposta
9. A+ <- V * Sigma+ * U^T
10. retorna A+
```

Listing 2.1: Pseudocódigo: pseudoinversa via SVD

O passo crítico é o limiar (*threshold*) da linha 2. Valores singulares abaixo dele são tratados como nulos, o que define o *posto numérico* da matriz. Esse limiar faz, na pseudoinversa, um papel parecido com o do critério de tolerância dos métodos iterativos: decide quando uma componente é “pequena o suficiente” para ser descartada e, com isso, mantém a estabilidade da

solução. A escolha usual é $\varepsilon = \max(m, n) \sigma_{\max} \varepsilon_{\text{maq}}$, onde $\varepsilon_{\text{maq}} \approx 2,22 \times 10^{-16}$ é a precisão de máquina (a mesma constante eps citada por Aquino).

2.3.2 Implementação em Python

A implementação foi feita em Python com a biblioteca NumPy, calculando A^+ a partir de `numpy.linalg.svd` – e, deliberadamente, *sem* recorrer a `numpy.linalg.pinv`, reservada à validação. O núcleo do código é reproduzido abaixo:

```

1 def pinv_svd(A, rcond=None, k=None):
2     A = np.asarray(A, dtype=float)
3     m, n = A.shape
4     U, s, Vt = np.linalg.svd(A, full_matrices=False) # A = U S V^T
5     if rcond is None:
6         rcond = max(m, n) * np.finfo(float).eps # limiar padrao
7     threshold = rcond * s[0]
8     s_inv = np.zeros_like(s)
9     mask = s > threshold # descarta sing. ~ 0
10    s_inv[mask] = 1.0 / s[mask]
11    A_plus = (Vt.T * s_inv) @ U.T # A+ = V Sigma+ U^T
12    return A_plus

```

Listing 2.2: Nucleo da implementacao (arquivo pseudoinversa.py)

No Scilab, ambiente do livro-texto, o mesmo resultado é obtido com os comandos `[U,S,V]=svd(A)` e, de forma pronta, `pinv(A)`, descrito por Aquino (2020) no capítulo de sistemas lineares. Quanto à complexidade, o custo é dominado pela SVD, $O(mn \min(m, n))$ em tempo, com armazenamento $O(mn)$; o tratamento de erros se concentra no descarte controlado dos valores singulares próximos de zero, evitando divisões por números muito pequenos que amplificariam o ruído.

2.4 Resultados e análise

2.4.1 Validação

A corretude da implementação foi verificada de duas formas. Primeiro, comparando A^+ com o resultado de `numpy.linalg.pinv`: a diferença, em norma de Frobenius, foi de $1,9 \times 10^{-17}$, ou seja, na ordem da precisão de máquina. Segundo, verificando numericamente as quatro condições (2.1) de Moore-Penrose para uma matriz 4×3 de posto 2 (deficiente em posto): todas as quatro foram satisfeitas, confirmando que a matriz calculada é, de fato, a pseudoinversa. A Tabela 2.1 resume o teste.

Tabela 2.1: Validação da implementação (matriz 4×3 de posto 2).

Verificação	Resultado
$\ A_{\text{manual}}^+ - A_{\text{numpy}}^+\ _F$	$1,9 \times 10^{-17}$
$AA^+A = A$	satisfeita
$A^+AA^+ = A^+$	satisfeita
$(AA^+)^T = AA^+$	satisfeita
$(A^+A)^T = A^+A$	satisfeita

2.4.2 Exemplo 1 – sistema sobredeterminado (4×2)

Para A de quatro linhas e duas colunas, com $b = (1, 2, 2, 4)^T$, a pseudoinversa retorna $x^* = A^+b = (0,9, 0,9)^T$, isto é, a reta de melhor ajuste $y = 0,9 + 0,9t$. O resíduo $\|Ax^* - b\|_2 = 0,8367$ é o menor possível, e verifica-se $A^T(Ax^* - b) = 0$, confirmando a condição de ortogonalidade dos mínimos quadrados (o resíduo é ortogonal ao espaço coluna de A).

2.4.3 Exemplo 2 – sistema subdeterminado (2×4)

Para A de duas linhas e quatro colunas com $b = (6, 16)^T$, obtém-se $x^* = (1,2, 1,4, 1,6, 1,8)^T$, que resolve o sistema exatamente ($\|Ax^* - b\|_2 \approx 3,6 \times 10^{-15}$) e tem norma $\|x^*\|_2 = 3,033$. Qualquer outra solução, da forma $x^* + tv$ com v no espaço nulo de A , ainda resolve o sistema, mas com norma estritamente maior – o que confirma o caráter de *mínima norma* da solução fornecida pela pseudoinversa.

2.4.4 Análise de posto e erro: SVD truncada

Como a pseudoinversa não é um método iterativo, não há tabelas de iterações nem gráficos de convergência no sentido usual. O papel análogo é desempenhado pela *SVD truncada*: retêm-se apenas os k maiores valores singulares no cálculo de A^+ e observa-se como o erro de reconstrução $\|Ax_k - b\|_2$ decresce à medida que k aumenta. A Tabela 2.2 e a Figura 2.1 apresentam esse comportamento para uma matriz 12×6 com espectro decrescente.

Tabela 2.2: Erro de reconstrução em função do posto k da SVD truncada.

k	σ_k	$\ Ax_k - b\ _2$	$\ x_k - x_{\text{real}}\ _2$	$\ x_k\ _2$
1	10,00	19,1403	3,6936	1,4489
2	6,00	6,2561	2,1319	3,3449
3	3,00	0,2992	0,4522	3,9405
4	1,20	0,0903	0,3910	3,9476
5	0,40	0,0253	0,3056	3,9536
6	0,08	0,0144	0,0541	3,9621

O resultado mostra um ponto importante: os primeiros valores singulares carregam quase toda a informação útil, enquanto os últimos, pequenos, pouco acrescentam à reconstrução e,

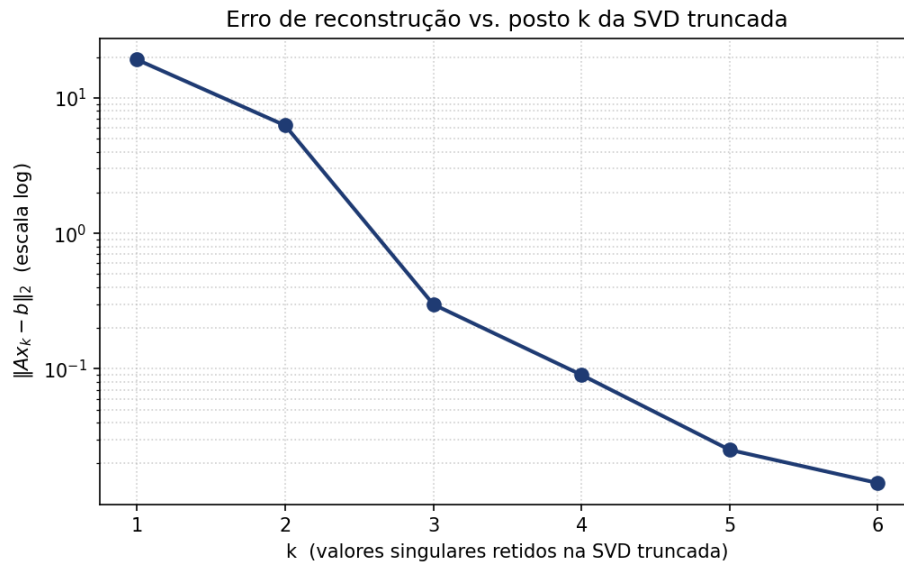


Figura 2.1: Erro de reconstrução $\|Ax_k - b\|_2$ em escala logarítmica versus o número k de valores singulares retidos. O erro cai rapidamente ao se incluírem os componentes dominantes.

quando há ruído nos dados, podem até atrapalhar. É esse o princípio da SVD truncada como técnica de *regularização*, justificado pelo teorema de Eckart-Young (1936), segundo o qual a aproximação de posto k é a melhor possível na norma considerada.

2.4.5 Análise do número de condição

Por fim, examinou-se o impacto do condicionamento sobre a estabilidade numérica. Geraram-se matrizes com $\kappa(A)$ variando de 1 a 10^{10} , perturbando-se o vetor b com um erro relativo fixo de 10^{-8} e medindo-se o erro relativo resultante na solução. A Figura 2.2 mostra que o erro na solução cresce proporcionalmente a $\kappa(A)$, acompanhando o limite teórico $\kappa(A) \cdot \delta$: para $\kappa(A) = 10^8$, um erro de 10^{-8} na entrada já produz erro da ordem de 40% na saída.

Na prática, isso quer dizer que, quando $\kappa(A)$ é grande, nem o cálculo exato da pseudoinversa protege contra a amplificação dos erros já presentes nos dados. Aqui a SVD truncada ajuda: ao descartar os valores singulares minúsculos, ela troca um pouco de viés por uma boa redução de variância.

2.5 Aplicações em Engenharia de Computação e Telecomunicações

2.5.1 Computação Gráfica: ajuste de curvas e compressão por SVD

Em Computação Gráfica, a pseudoinversa aparece sempre que se quer ajustar um modelo a um conjunto de pontos. O ajuste polinomial por mínimos quadrados é o exemplo direto: dados pontos (x_i, y_i) , monta-se a matriz de Vandermonde V e resolve-se $Vc = y$ por $c = V^+y$, obtendo os coeficientes do polinômio que melhor aproxima os dados. A Figura 2.3 mostra o ajuste de um

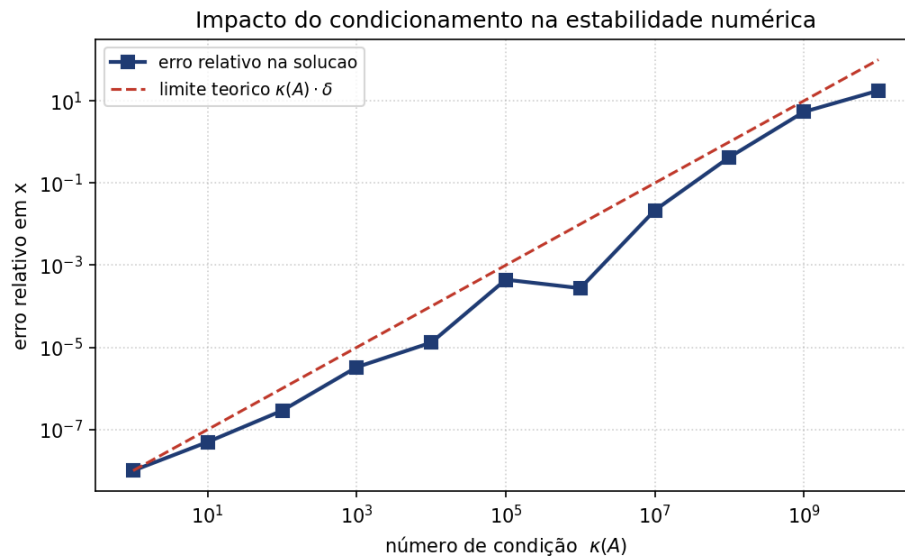


Figura 2.2: Erro relativo na solução em função de $\kappa(A)$ (escala log-log). O erro acompanha o limite teórico $\kappa(A) \cdot \delta$, o que mostra a perda de estabilidade em matrizes mal condicionadas.

polinômio de grau 5 a 18 pontos ruidosos, com erro quadrático médio de 0,168.

O mesmo princípio vale para a compressão de imagens: uma imagem, vista como matriz, pode ser aproximada por sua SVD truncada de posto k , guardando só os componentes mais importantes, que é a operação por trás da Tabela 2.2. A pseudoinversa também aparece na calibração de câmeras e na estimativa de transformações geométricas (*homografias*), em que um sistema sobredeterminado de correspondências de pontos é resolvido por mínimos quadrados.

2.5.2 Telecomunicações: estimacão de canal MIMO

Em sistemas MIMO (*Multiple-Input Multiple-Output*), o sinal recebido modela-se como $y = Hx + n$, onde H é a matriz de canal. A recuperação do sinal transmitido por um receptor de forçagem a zero (*zero-forcing*) usa diretamente a pseudoinversa, $\hat{x} = H^+y$, que minimiza o erro quadrático quando o número de antenas receptoras difere do de transmissoras. O número de condição de H indica, nesse contexto, a confiabilidade da estimativa – canais mal condicionados amplificam o ruído, conforme demonstrado na Figura 2.2.

2.5.3 Aprendizado de máquina: regressão linear

Na regressão linear, o vetor de parâmetros que minimiza o erro quadrático é precisamente $\theta = X^+y$, onde X é a matriz de características. A pseudoinversa fornece a solução de forma fechada, e sua versão regularizada (regressão *ridge*) equivale a amortecer os valores singulares pequenos, a mesma ideia de estabilização da seção anterior. Um conceito de 1920, portanto, está dentro de algoritmos de aprendizado de máquina de uso diário.

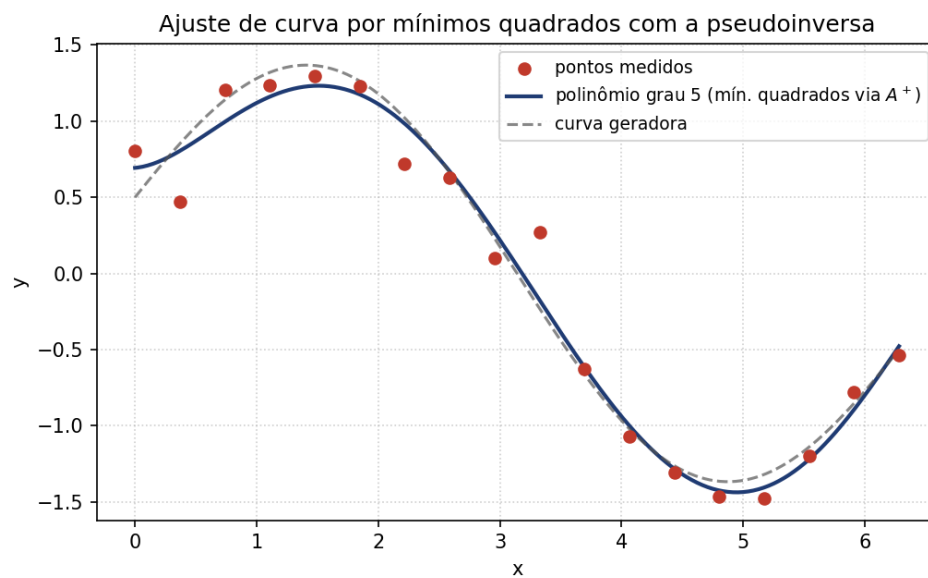


Figura 2.3: Ajuste de curva por mínimos quadrados via pseudoinversa: pontos medidos, polinômio ajustado ($c = V^+y$) e curva geradora.

3 Reflexão: cálculo numérico, computação e engenharia

O estudo da pseudoinversa deixa ver uma ideia mais geral: o cálculo numérico e a ciência da computação moderna não se separam. A definição de A^+ existe desde 1920, mas ela só virou ferramenta prática quando a computação ofereceu um algoritmo estável, a SVD, e o hardware para executá-lo em escala. História e técnica andam juntas: cada novo método costuma nascer de uma necessidade computacional e, depois de pronto, amplia o que a engenharia consegue fazer.

Para o engenheiro de computação e de telecomunicações, isso exige alguns conceitos que este trabalho ilustrou na prática. Um é a noção de *erro* (absoluto, relativo, de arredondamento e de truncamento). Como lembra Aquino (2020), uma pequena perturbação pode ser amplificada até levar a uma resposta sem sentido, e o número de condição é o que mede esse risco. Outro é a *estabilidade*: vimos que a forma via SVD da pseudoinversa é melhor que a equação normal por ser mais estável, embora as duas sejam “corretas” no papel. O terceiro é a *escolha do método*: não adianta funcionar só na teoria; é preciso escolher a variante numericamente segura para o problema concreto.

Esse estudo também deixou clara a importância de validar resultados por mais de um caminho. Verificar as quatro condições de Moore-Penrose e confrontar a implementação própria com uma rotina consagrada não é mero formalismo: é a aplicação prática do princípio, defendido por Aquino (2020), de que os erros devem ser minimizados ou, ao menos, identificados, empregando técnicas distintas para que os resultados sejam comparados. A regularização pela SVD truncada, por sua vez, ilustra um compromisso recorrente na engenharia: aceitar um pequeno viés em troca de uma boa redução da sensibilidade ao ruído.

Há, ainda, uma reflexão sobre o uso de bibliotecas prontas *versus* a implementação própria. Ferramentas como `pinv` do Scilab ou do NumPy resolvem o problema com uma única linha, e é razoável usá-las na maior parte do tempo. Mas tratá-las como “caixas pretas” tem riscos: a escolha do limiar de corte dos valores singulares, por exemplo, afeta silenciosamente o resultado, e um usuário desavisado pode não perceber que sua matriz é mal condicionada. Implementar a pseudoinversa do zero, como se fez aqui, revela essas decisões internas e forma o julgamento necessário para usar as ferramentas prontas com responsabilidade.

Por fim, a computação continua influenciando os métodos numéricos. A computação de alto desempenho, as GPUs e, mais à frente, a computação quântica mudam o tempo todo o que é viável calcular. Boa parte do avanço recente da inteligência artificial é, no fundo, métodos numéricos conhecidos (otimização, álgebra linear, regularização) rodando em hardware cada vez mais potente. O cálculo numérico não é um assunto encerrado; segue muito ativo.

Conclusão

Este trabalho analisou a história do cálculo numérico e sua relação com a ciência da computação, com foco na pseudoinversa de Moore-Penrose. Voltando aos objetivos propostos, percorremos marcos da disciplina, de Arquimedes aos ambientes atuais como o Scilab e o Python, e vimos como os computadores digitais transformaram métodos antes impraticáveis em ferramentas de uso corrente.

No tema específico, mostrou-se que a pseudoinversa generaliza a inversa clássica para matrizes quaisquer, sendo caracterizada de forma única pelas quatro condições de Moore-Penrose e calculada de modo estável pela decomposição em valores singulares. A implementação desenvolvida reproduziu A^+ a partir da SVD com erro da ordem da precisão de máquina em relação à rotina de referência, satisfaz as quatro condições e resolveu corretamente os casos sobredeterminado (mínimos quadrados) e subdeterminado (mínima norma). As análises de posto e de condicionamento explicitaram o papel do limiar dos valores singulares como critério de estabilidade (análogo ao critério de tolerância dos métodos iterativos) e o impacto do número de condição sobre a propagação de erros.

As aplicações mostraram a importância do tema para a formação do engenheiro: ajuste de curvas e compressão de imagens em Computação Gráfica, estimação de canal em sistemas MIMO e regressão em aprendizado de máquina, todas apoiadas na mesma ferramenta. O aprendizado principal é simples: entender o que está por baixo da função de biblioteca, e não apenas chamá-la, é o que permite usá-la com segurança. Para frente, a computação de alto desempenho, as GPUs e, possivelmente, a computação quântica devem manter o cálculo numérico no centro dos problemas de álgebra linear de larga escala.

Referências

- AQUINO, F. J. A. de. **Tópicos de métodos numéricos com Scilab**: computação científica para engenheiros. Rio de Janeiro: PoD Editora, 2020.
- BURDEN, R. L.; FAIRES, J. D. **Numerical Analysis**. 10. ed. Boston: Cengage Learning, 2015.
- CHAPRA, S. C.; CANALE, R. P. **Métodos numéricos para engenharia**. 7. ed. Porto Alegre: AMGH, 2016.
- ECKART, C.; YOUNG, G. The approximation of one matrix by another of lower rank. **Psychometrika**, v. 1, n. 3, p. 211–218, 1936.
- GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.
- MOORE, E. H. On the reciprocal of the general algebraic matrix. **Bulletin of the American Mathematical Society**, v. 26, p. 394–395, 1920.
- NUMPY DEVELOPERS. **numpy.linalg.pinv** — NumPy Manual. Disponível em: <https://numpy.org/doc/stable/>. Acesso em: 19 jun. 2026.
- PENROSE, R. A generalized inverse for matrices. **Mathematical Proceedings of the Cambridge Philosophical Society**, v. 51, n. 3, p. 406–413, 1955.
- STRANG, G. **Linear Algebra and Its Applications**. 4. ed. Belmont: Thomson Brooks/Cole, 2006.